

Fourier Transform Project 01

Introduction

The Fourier Transform is a very useful mathematical tool with numerous applications. It is a method by which the multitude of different frequencies contained within complex signals in the time domain are broken down into their individual constituents. (Some would say that, a chaotic signal with no obvious patterns in the time domain appears to "find harmony" in the frequency domain.) It takes what seems transient and frenetic—waves of sound, patterns of light, vibrations in machinery—and unveils their underlying structure. The tool is so versatile that it has found a place across disciplines as varied as telecommunications, mechanical diagnostics, and quantum mechanics. In the text below, four fundamental applications of Fourier Transform are explored. Examples are provided which hopefully help to underscore the foundational importance and transformational impact that applications of the Fourier Transform (and its inverse) have had across a wide variety of different fields of engineering and science.

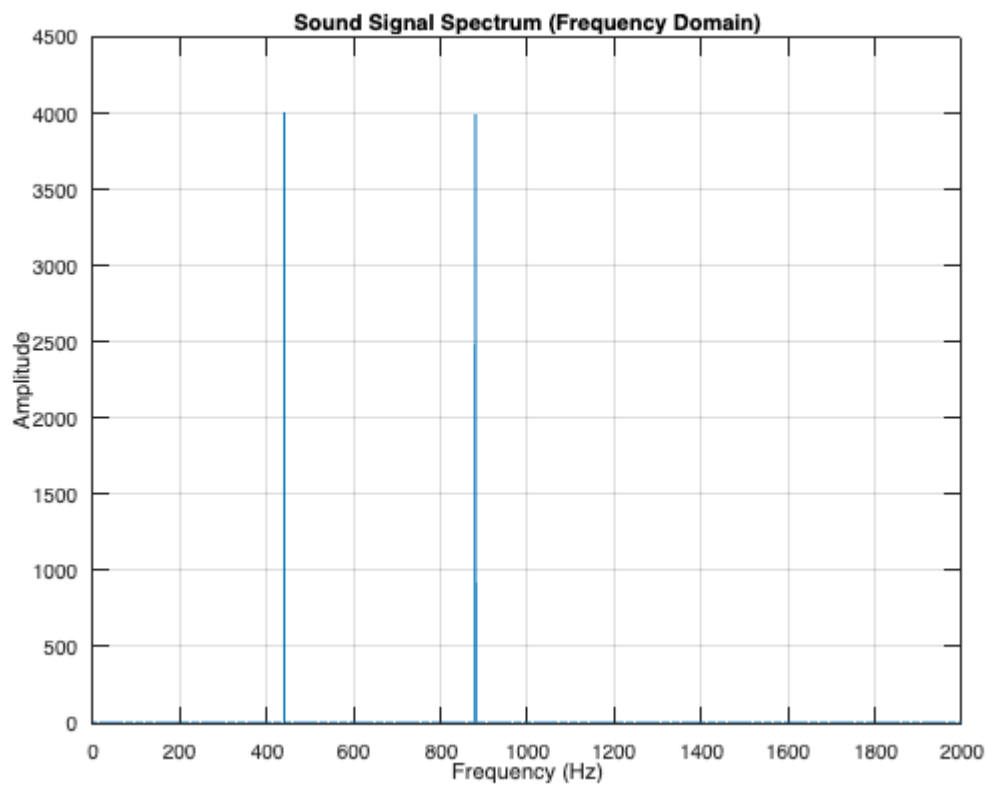
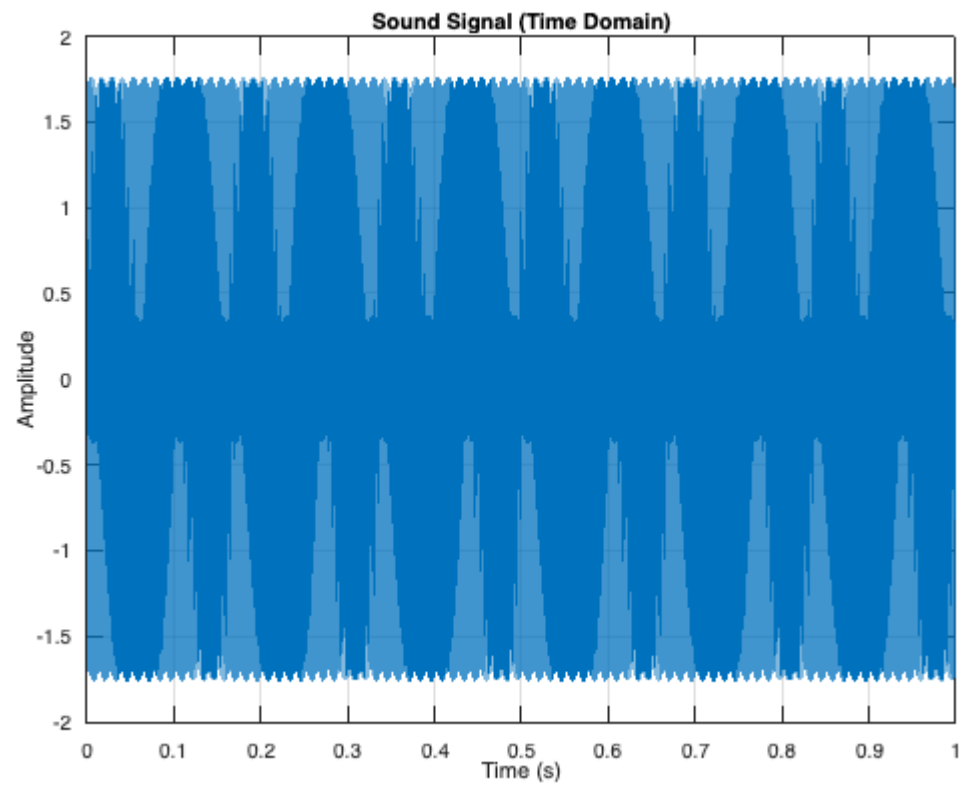
1.

Signal Analysis

Consider for a moment the nature of sound: a continuous vibration in the air, fleeting and ephemeral. Fourier Transform transcends this impermanence by rendering sound into a visual map of frequencies, amplitude, and phase. It is through this lens that the complexity of sound can be analyzed, understood, and reshaped.

In speech recognition systems, Fourier Transform is used to analyze human speech by converting it into the frequency domain, where phonemes—distinct units of sound—emerge with clarity. For example, a person's utterance of the word "hello" produces a unique spectral fingerprint. Fourier-based analysis enables systems like Alexa or Siri to identify these patterns amid a cacophony of background noise.

MATLAB Example: Audio Frequency Spectrum Analysis

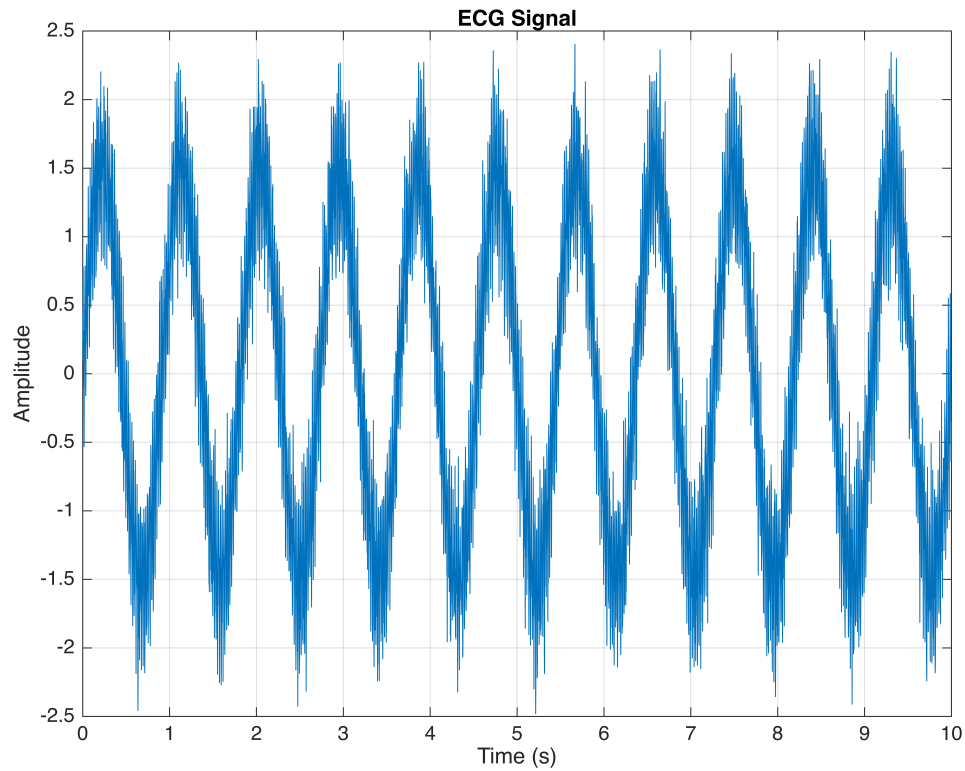


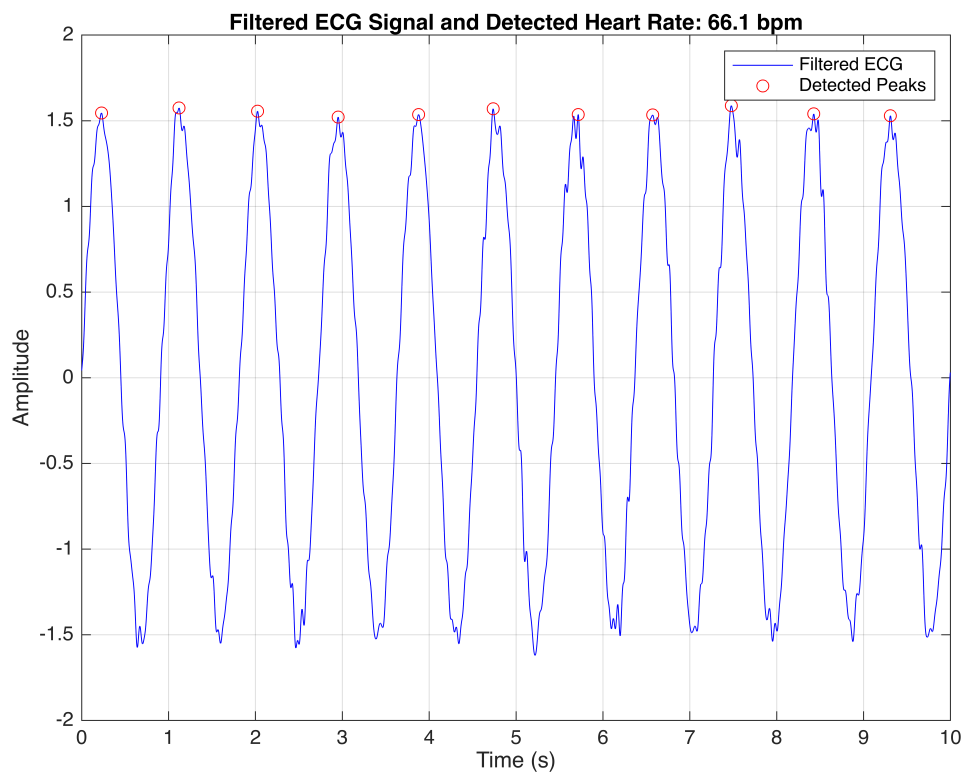
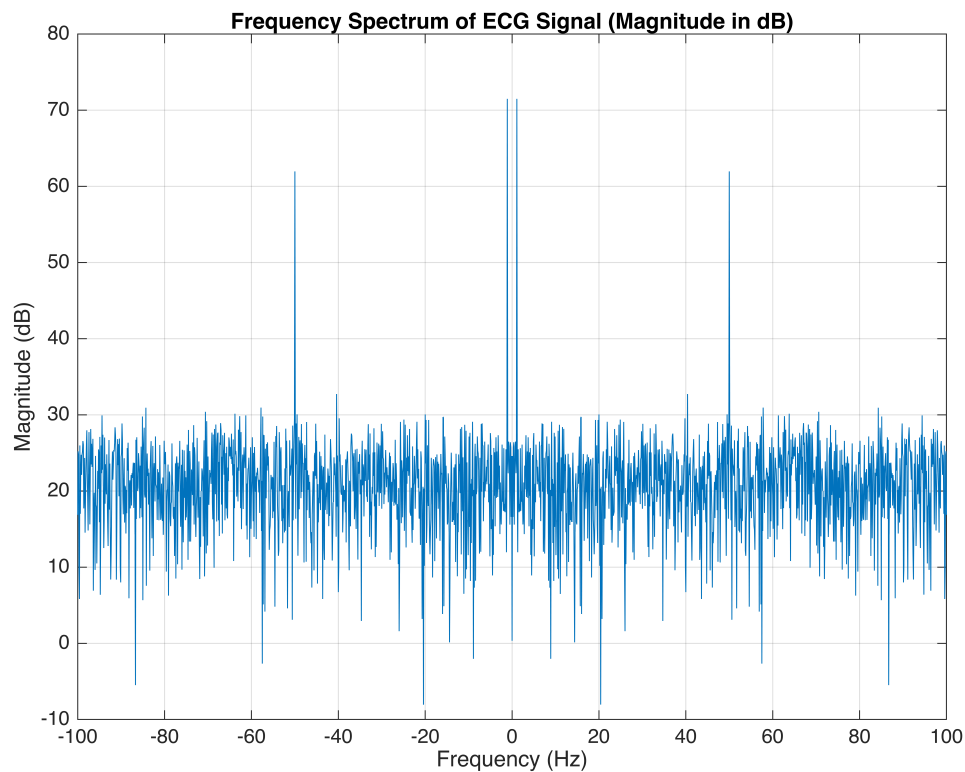
Source: <https://www.mathworks.com/help/matlab/math/basic-spectral-analysis.html>

ECG Signal Analysis

In another domain, biomedical signal processing, the Fourier Transform plays an important role. Electrocardiogram (ECG) signals, for example, are decomposed to identify irregular heartbeats or arrhythmias. A Fourier analysis of such signals can separate normal periodic rhythms from anomalies, offering clinicians a non-invasive diagnostic tool.

MATLAB Example: ECG Signal Analysis





- This highlights how the Fourier Transform decomposes a signal into its frequency components for analysis and filtering.
- By removing unwanted high-frequency noise in the frequency domain, the inverse transform reconstructs a cleaner ECG signal for further analysis.

Sample data from: https://github.com/mathworks/physionet_ECG_data

MATLAB Code sources found here: <https://www.mathworks.com/help/matlab/math/fourier-transforms.html>

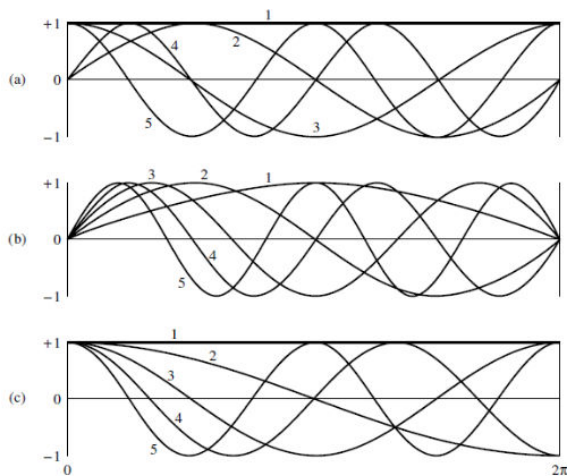
Image Processing

Where sound is fleeting, an image is static—yet it too hides a deeper reality. Images are not mere collections of pixels but are layered with frequencies that represent edges, gradients, and details. The Fourier Transform deciphers these components, facilitating tasks such as filtering, compression, and feature extraction.

Take image compression in the JPEG format as a case study. By transforming an image into the frequency domain using the Discrete Cosine Transform (a close relative of Fourier Transform), high-frequency components—those that the human eye is less sensitive to—can be selectively reduced. This results in substantial compression without perceptible loss of quality.

D.C.T. vs F.T. Comparison:

- Only the real part of FFT (less data overheads).
- Computationally simpler than FFT.
- DCT— effective for multimedia compression (energy compaction).
- DCT much more commonly used (than FFT) in multimedia image/video compression — more later.
- Cheap MPEG Audio variant — more later.
- FT captures more frequency “fidelity” (e.g . phase).

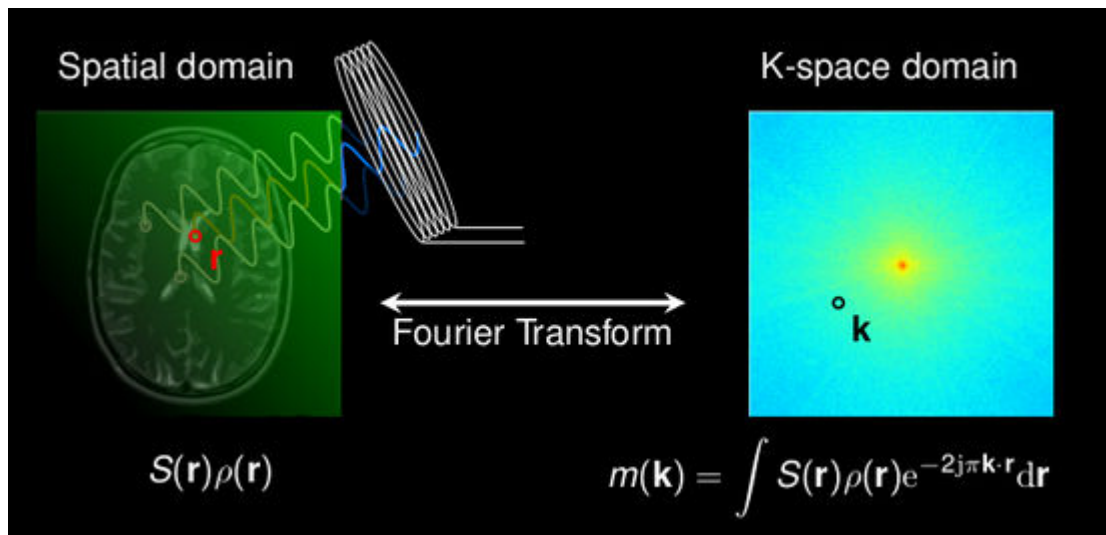


(a) Fourier transform, (b) Sine transform, (c) Cosine transform.

MRI Reconstruction

Another application lies in medical imaging. In magnetic resonance imaging (MRI), Fourier Transform enables the reconstruction of spatial information from raw data collected in the frequency domain. Without it, the detailed anatomical structures revealed in an MRI scan would remain hidden in chaos.

MRI Reconstruction Example



Source: <https://bigwww.epfl.ch/algorithms/mri-reconstruction/>

3.

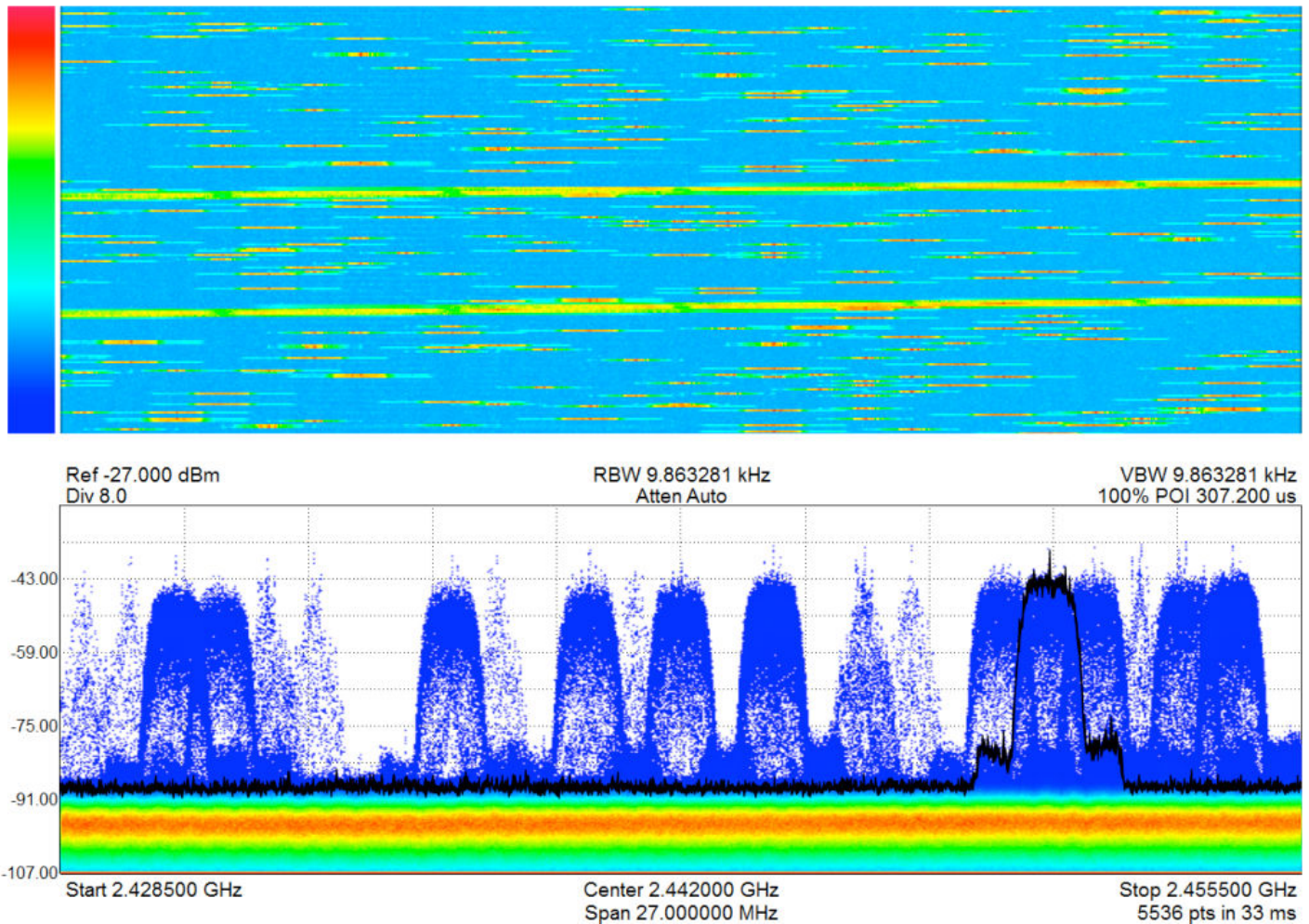
Software-Defined Radio

In a world increasingly dominated by wireless communication, Software-Defined Radio (SDR) has emerged as a transformative technology. At the heart of SDR lies the Fourier Transform, a tool that allows digital systems to interact with analog waves in the electromagnetic spectrum.

Imagine a crowded urban environment where numerous devices—smartphones, routers, and wearables—compete for limited radio bandwidth. An SDR equipped with Fourier Transform can dynamically scan the spectrum, identify unused channels, and switch frequencies in real-time. This capability, known as dynamic spectrum access, enhances communication efficiency and reduces interference.

Consider the defense industry, where SDRs are deployed in military communications. By leveraging Fourier Transform, these radios can rapidly switch frequencies, a practice called frequency hopping, to evade jamming attempts by adversaries. Fourier analysis ensures such transitions are seamless, preserving the integrity of transmitted information.

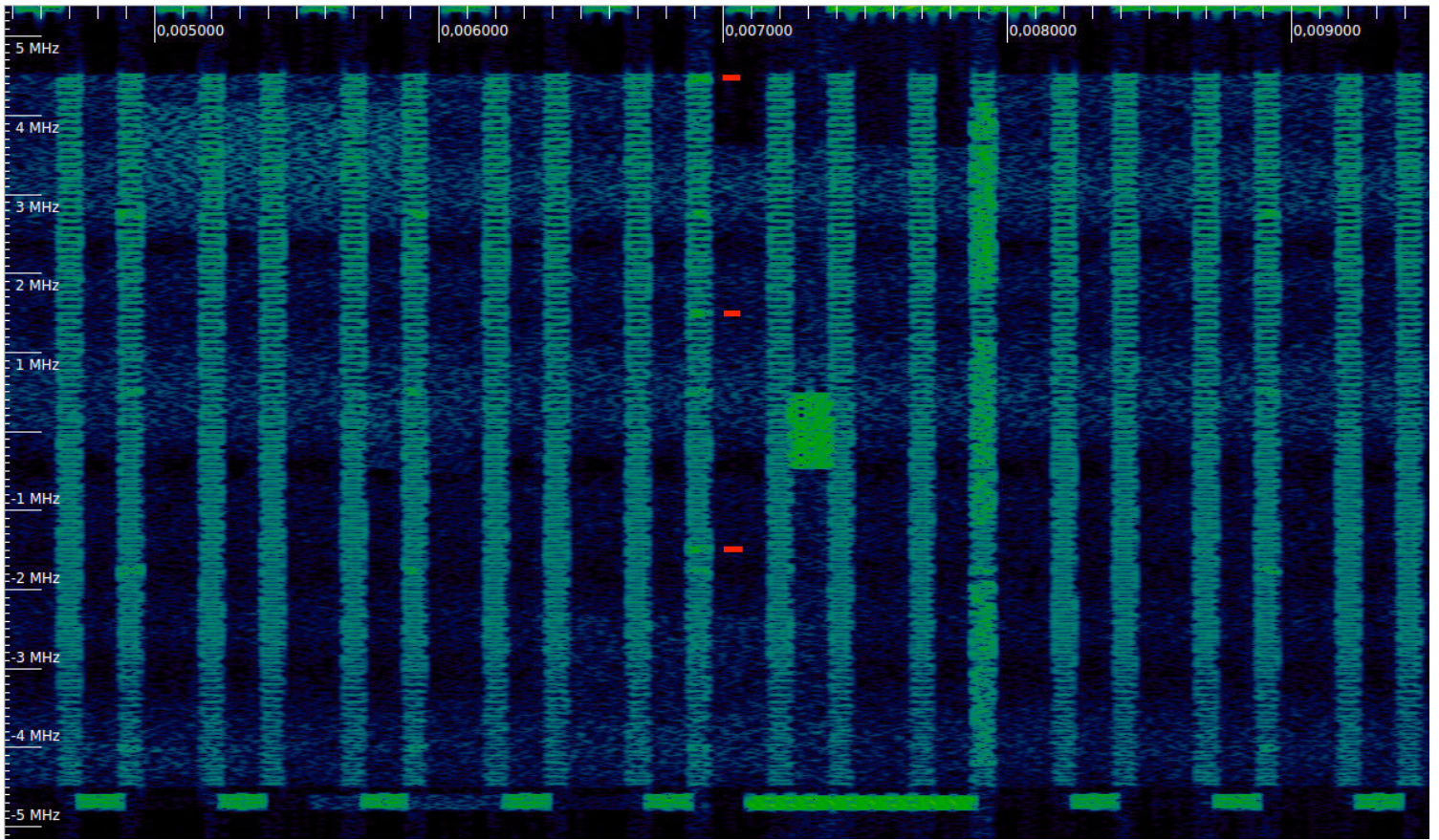
Image of the EM Spectrum spanning 2.4285 GHz to 2.4555 GHz



The lower plot contains a frequency domain representation of the em spectrum, the peaks are likely different signals from wi-fi devices passing through the air, which are represented over time along the y-axis in the water fall plot above it.

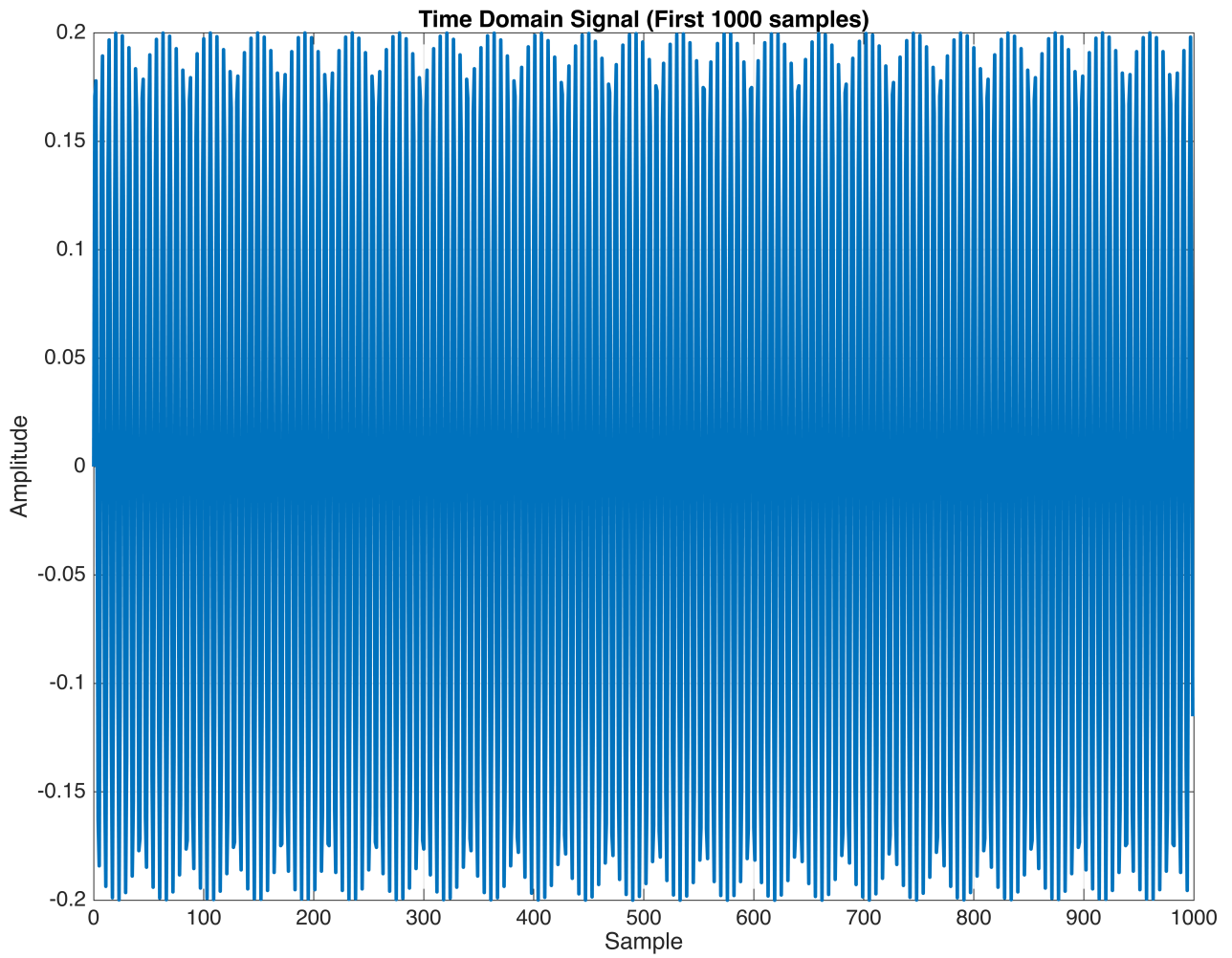
Image Source: <https://signalhound.com/wp-content/uploads/2016/03/rtsa-display-1024x725.png>

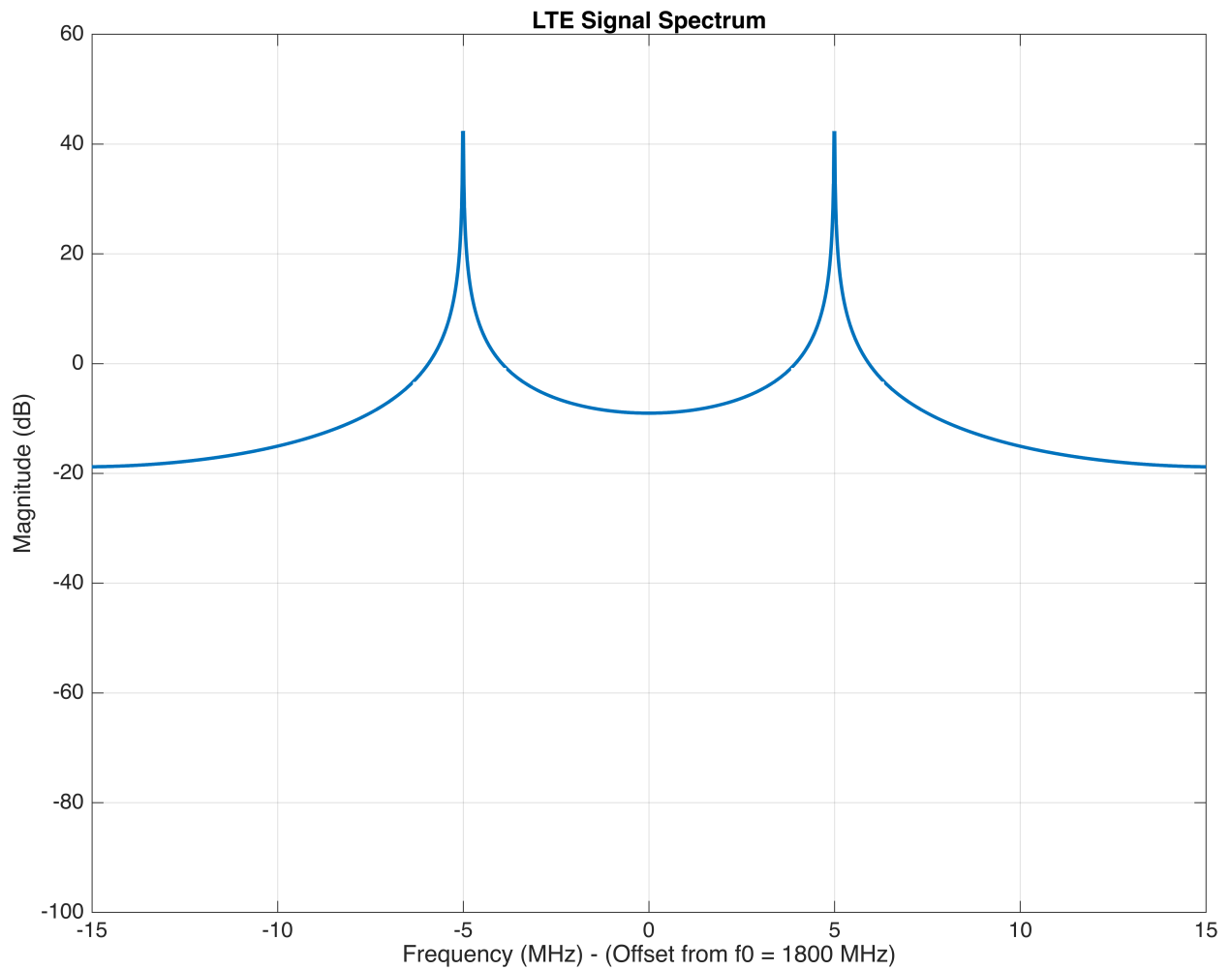
LTE Downlink Spectrum Waterfall:



Source for above image https://desteveez.net/wp-content/uploads/2022/06/phich_waterfall.jpg

LTE Frequency Spectrum MATLAB Example





Source for the LTE Band 3 Parameters used in the code above:

<https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=2411>

Sources for MATLAB code:

<https://www.mathworks.com/help/comm/gs/introduction-to-ofdm.html>

<https://www.mathworks.com/content/dam/mathworks/mathworks-dot-com/solutions/automotive/files/in-expo-2012/modelling-a-4g-lte-system-in-matlab.pdf>

MATLAB Code Used To Make This Project:

Section One:

```
% MATLAB Example: Demonstration of the Fourier Transform on a sound signal
% This code generates 1 second of two musical notes:
% A4: 440(Hz)
%      and
% A5: 880 (Hz)
%-----|
% generate notes
fsamplerate = 8000;
t = 0:1/fsamplerate:1-1/fsamplerate;
freq1 = 440;
freq2 = 880;
soundSignal = sin(2*pi*freq1*t) + sin(2*pi*freq2*t);
%-----|
% apply the Fourier Transform
n = length(soundSignal);
freq = (0:n-1)*(fsamplerate/n);
Y = fft(soundSignal);
%----- RESULTS: -----|
% Plot 1: time domain
figure;
plot(t, soundSignal);
title('Sound Signal (Time Domain)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
%-----|
% Plot 2: frequency domain
figure;
plot(freq, abs(Y));
title('Sound Signal Spectrum (Frequency Domain)');
xlabel('Frequency (Hz)');
ylabel('Amplitude');
xlim([0 2000]);
grid on;
```

MATLAB Example: ECG Signal Analysis

```
% Fourier Transform example:
% *** ECG data frequency content analysis ***

fsamplerate = 500;
t = 0:1/fsamplerate:10-1/fsamplerate; % adds noise
ecgSignal = 1.5*sin(2*pi*1.1*t) + 0.5*sin(2*pi*50*t) + 0.2*randn(size(t));
n = length(ecgSignal);
freq = (0:n-1)*(fsamplerate/n);
Y = fft(ecgSignal);
```



```

% Shifts zero-frequency component to center of spectrum

Y_shifted = fftshift(Y);
freq_shifted = (-n/2:n/2-1)*(fsamplerate/n);
magnitude_dB = 20 * log10(abs(Y_shifted) + eps); % eps avoids log(0) errors

% Frequency-domain filter is applied in order
% to filter out high frequencies:
% Uses a 20 Hz cutoff
cutoff = 20;
Y_filtered = Y;
Y_filtered(abs(freq) > cutoff) = 0;

% Apply an Inverse Fourier Transform to
% reconstruct time-domain signal:

filteredECG = ifft(Y_filtered, 'symmetric');

% peak detection (0.6 s apart at minimum)

[peaks, locs] = findpeaks(filteredECG, 'MinPeakDistance', 0.6 *
fsamplerate);
if length(locs) > 1
    heartRate = 60 / mean(diff(locs) / fsamplerate);
else
    heartRate = NaN;
end

% Plot 1 – Original signal in time domain:

figure;
plot(t, ecgSignal);
title('ECG Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

```

Section Three:

```

% Standard LTE sampling rate @ 30.72 MHz for
% a single LTE symbol duration:

samplingRate = 30.72e6;
timeWindow = 1/(15e3);
t = 0:1/samplingRate:timeWindow;

% LTE Band 3 Parameters (source linked below)

centerFreq = 1800e6;

```

```

subcarrierSpacing = 15e3;
numSubcarriers = 1200;

% OFDM signal generator function

function signal = generateOFDMSignal(cellFreq, power, t, centerFreq)
    signal = power * sin(2*pi*(cellFreq - centerFreq)*t);
end

% generates three LTE cells, starting with primary cell at center frequency

signal1 = generateOFDMSignal(centerFreq, 1.0, t, centerFreq);
signal2 = generateOFDMSignal(centerFreq + 5e6, 0.7, t, centerFreq); %
adjacent cell +5MHz
signal3 = generateOFDMSignal(centerFreq - 5e6, 0.5, t, centerFreq); %
adjacent cell -5MHz
lteSignal = signal1 + signal2 + signal3; %
combines signals
ft = fftshift(fft(lteSignal));
ftMag = abs(ft);
freqAxis = (-length(ft)/2:length(ft)/2-1)*(samplingRate/length(ft))/1e6;

% time domain plot

figure('Position', [100, 100, 800, 600])
plot(t(1:1000)*samplingRate, real(lteSignal(1:1000)), 'LineWidth', 2)
grid on
title('Time Domain Signal (First 1000 samples)')
xlabel('Sample')
ylabel('Amplitude')
set(gca, 'FontSize', 12)

% frequency domain plot in dB

ftShift_dB = 20 * log10(ftMag);
figure('Position', [200, 200, 800, 600])
plot(freqAxis, ftShift_dB, 'LineWidth', 2)
grid on
title('LTE Signal Spectrum')
xlabel('Frequency (MHz) - (Offset from f0 = 1800 MHz)')
ylabel('Magnitude (dB)')
xlim([-15 15])
ylim([-100 60])
set(gca, 'FontSize', 12)

```

Section Four:

% Example MATLAB code for simulating bridge mechanical resonances:

```
Fs = 500;
```

```

T = 1/Fs;
L = 1500;
t = (0:L-1)*T;

% Parameters for the simulation of wind, traffic, & structural vibrations:

f_wind = 10; % Wind-induced vibration frequency (Hz)
f_traffic = 30; % Traffic-induced vibration frequency (Hz)
f_fatigue = 70; % Frequency indicating structural fatigue (Hz)
signal = 1*sin(2*pi*f_wind*t) + 0.5*sin(2*pi*f_traffic*t) + ...
        0.2*sin(2*pi*f_fatigue*t) + 0.1*randn(size(t));

% This section computes the Fourier Transform of the signal:

Y = fft(signal);
P2 = abs(Y/L);
P1 = P2(1:L/2+1);
P1(2:end-1) = 2*P1(2:end-1);
f = Fs*(0:(L/2))/L;

%-----|
% Time-domain Plot

figure;
plot(t, signal);
title('Bridge Vibration Signal');
xlabel('Time (s)');
ylabel('Amplitude');

%-----|
% Frequency-domain Plot

figure;
plot(f, P1);
title('Frequency Spectrum');
xlabel('Frequency (Hz)');
ylabel('|P1(f)|');
grid on;

```

All Works Cited:

Section One:

<https://www.mathworks.com/help/matlab/math/basic-spectral-analysis.html>

https://github.com/mathworks/physionet_ECG_data

<https://www.mathworks.com/help/matlab/math/fourier-transforms.html>

Section Two:

<https://users.dimi.uniud.it/~antonio.dangelo/MMS/2013/lessons/L11lecture.pdf>

<https://bigwww.epfl.ch/algorithms/mri-reconstruction/>

Section Three:

<https://signalhound.com/wp-content/uploads/2016/03/rtsa-display-1024x725.png>

https://desteveez.net/wp-content/uploads/2022/06/phich_waterfall.jpg

<https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=2411>

<https://www.mathworks.com/help/comm/gs/introduction-to-ofdm.html>

<https://www.mathworks.com/content/dam/mathworks/mathworks-dot-com/solutions/automotive/files/in-expo-2012/modelling-a-4g-lte-system-in-matlab.pdf>

Section Four:

<https://endaq.com/pages/free-vibration-analysis-files>

<https://blog.endaq.com/vibration-analysis-fft-psd-and-spectrogram>

<https://www.mathworks.com/help/matlab/math/fourier-transforms.html>